Gramática de expresiones aritméticas asociativas a la derecha

1. Gramática

Gramática libre de contexto, no ambigua, en la que **todos los operadores son asociativos a la derecha**, con la precedencia habitual (`^` > `* /` > `+ -`):

```
E  →  T + E  |  T - E  |  T          (menor precedencia: + -)
T  →  F * T  |  F / T  |  F          (precedencia media: * /)
F  →  P ^ F  |  P                   (mayor precedencia: ^)
P  →  ( E )  |  num  |  id           (primarios / átomos)
```

- **Asociatividad a la derecha:** cada regla es **recursiva por la derecha**. El no terminal que se define (`E`, `T`, `F`) aparece a la *derecha* del operador, así que los operadores repetidos se anidan hacia la derecha.
- **Precedencia:** cada nivel solo genera el nivel inmediatamente superior a la izquierda de su operador, por lo que los operadores que ligan más fuerte quedan más profundos en el árbol.

2. Ejemplo: `8 - 3 - 2`

Derivación:

```
E ⇒ T - E
  ⇒ 8 - E
  ⇒ 8 - (T - E)
  ⇒ 8 - (3 - E)
  ⇒ 8 - (3 - 2)        = 8 - 1 = 7
```

Árbol de derivación:

```
      E
    / | \
   T  -  E
   |   / | \
   8  T  -  E
      |     |
      3     2
```

Resultado: `8 - (3 - 2) = 7`. Con una gramática asociativa a la izquierda sería `(8 - 3) - 2 = 3`.

Precedencia mezclada: `2 + 3 * 4 ^ 2`

Se analiza como `2 + (3 * (4 ^ 2))`. El `^` queda en lo más profundo porque está en el nivel más bajo de la gramática.

Doble resta: `10 - 4 - 3`

Es el caso en el que más se nota la asociatividad, porque la resta no es asociativa: según cómo se agrupe, el resultado cambia.

Derivación (cada `E` se expande con `E → T - E`, y la última con `E → T`):

```
E ⇒ T - E
  ⇒ 10 - E
  ⇒ 10 - (T - E)
  ⇒ 10 - (4 - E)
  ⇒ 10 - (4 - T)
  ⇒ 10 - (4 - 3)
```

Árbol de sintaxis abstracta que produce el parser:

```
        -
       / \
     10   -
         / \
        4   3

Op '-' (Num 10) (Op '-' (Num 4) (Num 3))
```

Comparación:

Asociatividad	Agrupación	Resultado
Derecha (esta gramática)	`10 - (4 - 3)`	**9**
Izquierda (convencional)	`(10 - 4) - 3`	3

Para obtener la agrupación por la izquierda con esta gramática hay que escribir los paréntesis explícitamente: `(10 - 4) - 3`.

Traza del parser en Haskell (`nivel "+-"` es la función que implementa `E`):

```
nivel "+-" [10 - 4 - 3]
├─ lee T = 10, ve '-', se llama a sí misma con el resto
│  nivel "+-" [4 - 3]
│  ├─ lee T = 4, ve '-', se llama a sí misma con el resto
│  │  nivel "+-" [3]
│  │  └─ lee T = 3, no hay operador → devuelve Num 3
│  └─ arma Op '-' (Num 4) (Num 3)
└─ arma Op '-' (Num 10) (Op '-' (Num 4) (Num 3))
```

La subexpresión derecha se termina de construir antes que la izquierda, y por eso la resta de la derecha queda más profunda en el árbol.

3. Versión LL(1)

La gramática anterior no es LL(1): alternativas como `T + E` y `T` empiezan con el mismo símbolo. Al factorizar por la izquierda se obtiene:

```
E   →  T E'
E'  →  + E  |  - E  |  ε
T   →  F T'
T'  →  * T  |  / T  |  ε
F   →  P F'
F'  →  ^ F  |  ε
P   →  ( E )  |  num  |  id
```

Genera el mismo lenguaje y sigue siendo asociativa a la derecha, porque `E'` vuelve a llamar a `E` (y no a `E'`).

> **Nota:** en matemáticas convencionales solo `^` es asociativo a la derecha. Para el comportamiento usual, `+ - * /` deberían ser recursivos por la izquierda (p. ej. `E → E + T | E - T | T`), manteniendo `F → P ^ F | P`.

4. Parser en Haskell

Parser descendente recursivo sin dependencias externas que implementa directamente la gramática LL(1). Cada nivel de precedencia lo resuelve la función `nivel`: tras leer un operador vuelve a llamarse a sí misma (recursión por la derecha), que es exactamente lo que da la asociatividad a la derecha.

El código también está en `Parser.hs`.

```haskell
-- Parser descendente recursivo para expresiones aritméticas
-- con todos los operadores asociativos a la derecha.
--
-- Gramática LL(1):
--   E  → T E'      E' → + E | - E | ε
--   T  → F T'      T' → * T | / T | ε
--   F  → P F'      F' → ^ F | ε
--   P  → ( E ) | num | id
module Main where

import Data.Char (isAlpha, isAlphaNum, isDigit, isSpace)

-- ──────────────── Analizador léxico ────────────────

data Token
  = TNum Integer
  | TId String
  | TOp Char
  | TParIzq
  | TParDer
  deriving (Show, Eq)

tokenizar :: String -> Either String [Token]
tokenizar [] = Right []
tokenizar s@(c : cs)
  | isSpace c = tokenizar cs
  | isDigit c = let (ds, resto) = span isDigit s in (TNum (read ds) :) <$> tokenizar resto
  | isAlpha c = let (xs, resto) = span isAlphaNum s in (TId xs :) <$> tokenizar resto
  | c `elem` "+-*/^" = (TOp c :) <$> tokenizar cs
  | c == '(' = (TParIzq :) <$> tokenizar cs
  | c == ')' = (TParDer :) <$> tokenizar cs
  | otherwise = Left ("carácter inesperado: " ++ show c)

-- ──────────────── Árbol de sintaxis ────────────────

data Expr
  = Num Integer
  | Var String
  | Op Char Expr Expr
  deriving (Show, Eq)

-- ──────────────── Analizador sintáctico ────────────────

-- Un parser consume tokens y devuelve el resultado junto con los tokens restantes.
type Parser a = [Token] -> Either String (a, [Token])
```

```
-- Un nivel de precedencia:  X → Y X'   con   X' → op X | ε
-- Tras el operador se llama de nuevo a X (no a Y): esa recursión
-- por la derecha es la que produce la asociatividad a la derecha.
nivel :: String -> Parser Expr -> Parser Expr
nivel ops siguiente ts = do
  (izq, resto) <- siguiente ts
  case resto of
    TOp op : resto' | op `elem` ops -> do
      (der, resto'') <- nivel ops siguiente resto'
      Right (Op op izq der, resto'')
    _ -> Right (izq, resto)

parseE, parseT, parseF, parseP :: Parser Expr
parseE = nivel "+-" parseT -- E → T E'
parseT = nivel "*/" parseF -- T → F T'
parseF = nivel "^" parseP -- F → P F'

-- P → ( E ) | num | id
parseP (TNum n : r) = Right (Num n, r)
parseP (TId x : r) = Right (Var x, r)
parseP (TParIzq : r) = do
  (e, r') <- parseE r
  case r' of
    TParDer : r'' -> Right (e, r'')
    _ -> Left "se esperaba ')'"
parseP (t : _) = Left ("token inesperado: " ++ show t)
parseP [] = Left "fin de entrada inesperado"

-- Analiza una cadena completa; falla si sobran tokens.
analizar :: String -> Either String Expr
analizar s = do
  ts <- tokenizar s
  (e, resto) <- parseE ts
  case resto of
    [] -> Right e
    t : _ -> Left ("token sobrante: " ++ show t)

-- ──────────── Utilidades ────────────

-- Muestra la expresión totalmente parentizada para ver la asociatividad.
mostrar :: Expr -> String
mostrar (Num n) = show n
mostrar (Var x) = x
mostrar (Op op a b) = "(" ++ mostrar a ++ " " ++ [op] ++ " " ++ mostrar b ++ ")"

-- Evalúa la expresión con un entorno de variables.
evaluar :: [(String, Double)] -> Expr -> Either String Double
```

```
evaluar _ (Num n) = Right (fromIntegral n)
evaluar env (Var x) = maybe (Left ("variable no definida: " ++ x)) Right (lookup x env)
evaluar env (Op op a b) = do
  x <- evaluar env a
  y <- evaluar env b
  case op of
    '+' -> Right (x + y)
    '-' -> Right (x - y)
    '*' -> Right (x * y)
    '/' | y == 0 -> Left "división por cero"
        | otherwise -> Right (x / y)
    '^' -> Right (x ** y)
    _ -> Left ("operador desconocido: " ++ [op])

main :: IO ()
main = mapM_ probar ejemplos
  where
    entorno = [("x", 2), ("y", 3)]
    ejemplos =
      [ "8 - 3 - 2"
      , "10 - 4 - 3"
      , "(10 - 4) - 3"
      , "16 / 4 / 2"
      , "2 ^ 3 ^ 2"
      , "2 + 3 * 4 ^ 2"
      , "(8 - 3) - 2"
      , "x * y - 1"
      , "2 + * 3"
      ]
    probar s = case analizar s of
      Left err -> putStrLn (s ++ "  =>  error: " ++ err)
      Right e ->
        putStrLn $
          s ++ "  =>  " ++ mostrar e ++ "  =  "
            ++ either ("error: " ++) show (evaluar entorno e)
```

Ejecución

```
runghc Parser.hs
```

Salida:

```
8 - 3 - 2  =>  (8 - (3 - 2))  =  7.0
10 - 4 - 3  =>  (10 - (4 - 3))  =  9.0
(10 - 4) - 3  =>  ((10 - 4) - 3)  =  3.0
16 / 4 / 2  =>  (16 / (4 / 2))  =  8.0
2 ^ 3 ^ 2  =>  (2 ^ (3 ^ 2))  =  512.0
2 + 3 * 4 ^ 2  =>  (2 + (3 * (4 ^ 2)))  =  50.0
(8 - 3) - 2  =>  ((8 - 3) - 2)  =  3.0
x * y - 1  =>  ((x * y) - 1)  =  5.0
2 + * 3  =>  error: token inesperado: TOp '*'
```

`mostrar` imprime la expresión con todos los paréntesis para que se vea cómo se agrupa: `8 - 3 - 2` queda como `(8 - (3 - 2))` y `2 ^ 3 ^ 2` como `(2 ^ (3 ^ 2))`.